ZYVAN

Product Requirements Document

Reliable webhook and event infrastructure for growing SaaS teams

Version 1.1 • 26 August 2026

PRODUCT THESIS

Zyvan should not compete as a generic “cheaper Hookdeck/Svix.” The initial wedge is a simple, production-grade reliability layer for SaaS and fintech teams that need durable webhook delivery without operating the queue/retry/replay system themselves. India is the first distribution wedge, not the entire product identity.

Document Control

Field	Value
Product	Zyvan
Version	1.0
Date	26 August 2026
Status	Proposed / Pre-PMF
Primary decision	Build a narrow, reliable webhook/event platform and validate with production customers before broad feature expansion
Primary ICP	5–100 person SaaS / API companies; initial focus on India-based fintech, SaaS and Shopify ecosystems

1. Executive Summary

Zyvan is a developer infrastructure platform for reliable event and webhook delivery. It ingests events, persists them durably, queues work asynchronously, delivers to HTTP endpoints, retries transient failures, isolates failed deliveries, and provides replay and operational visibility.

The core business problem is not “sending an HTTP POST.” The real problem is operating webhook delivery when events are duplicated, endpoints fail, traffic spikes, jobs run long, tenants have uneven workloads, and engineers need to reconstruct what happened after a failure.

Zyvan is entering a validated but crowded category. Hookdeck, Svix, Convoy and other platforms already cover much of the generic webhook gateway surface. Therefore, this PRD explicitly avoids a feature-count race. The product strategy is to win a focused segment through simplicity, developer experience, India-first distribution, strong provider integrations, and multi-tenant reliability controls.

2. Problem Statement

2.1 Customer pain

- Webhook providers may retry, duplicate, delay or deliver events out of order.
- Customer endpoints may return 4xx/5xx responses, time out, rate-limit, or temporarily disappear.
- Large tenants can consume worker capacity and increase latency for smaller tenants (“noisy neighbor” problem).
- Teams often build queues, retry state, idempotency, dead-letter handling, replay tools and dashboards internally.
- Debugging requires correlating provider logs, application logs, queue state and HTTP responses.
- For payment and billing workflows, a lost or delayed event can become a real customer-support and revenue problem.

2.2 Jobs to be done

- When an important event occurs, I want it to be durably accepted so I do not lose it if my downstream system is unavailable.
- When delivery fails, I want retries to happen automatically without writing retry infrastructure myself.
- When something goes wrong, I want to see exactly what happened and replay the event safely.
- When one tenant generates a spike, I want other tenants to remain healthy.
- When I expose webhooks to my customers, I want secure signatures, predictable semantics and an operational audit trail.

3. Product Goals

Goal	Definition of success
Reliable delivery	No event is silently discarded inside the configured retention window; all delivery attempts have durable state.
Fast integration	A developer can send the first event in under 15 minutes using API docs/SDK examples.
Operational clarity	Every delivery has a human-readable timeline, response details, retry state and replay action.
Tenant isolation	One tenant cannot monopolize worker capacity or materially degrade other tenants.
Commercial viability	Paid plans have predictable unit economics and usage-based expansion.
Customer validation	Within 12–16 weeks, secure at least 3 production customers and evidence of willingness to pay.

3.1 Non-goals for v1

- Competing feature-for-feature with Hookdeck, Svix or Convoy.
- Building a general-purpose workflow engine comparable to Inngest or Trigger.dev.
- Supporting Kafka/SQS/PubSub as first-class internal brokers on day one.
- Enterprise compliance certifications before product-market signals exist.
- Multi-region active-active infrastructure before the single-region system is proven reliable.
- A broad transformation/routing language in the first release.

4. Positioning & Product Strategy

4.1 Proposed positioning

Recommended positioning:
“Reliable webhook infrastructure for growing SaaS companies.”

Supporting statement:
“Zyvan receives, stores, retries, delivers and replays critical webhooks so your team does not have to build and operate that infrastructure.”

4.2 Wedge

- Primary wedge: 5–100 person SaaS/API companies that are too serious for ad-hoc webhook code but too small to maintain a dedicated reliability platform.
- Initial distribution wedge: India-based SaaS, fintech, Shopify apps and developer-tool startups.
- Product wedge: simple reliability + operational debugging + tenant isolation rather than an enormous gateway feature set.

4.3 Competitive posture

Competitor	What they prove	Zyvan response
Hookdeck	Demand for managed event gateways, outbound delivery, multi-tenancy, observability, routing and enterprise controls. Current Event Gateway pricing shown at $0 free, $39 Team and $499 Growth.	Do not clone the full platform. Win on simplicity, startup affordability, India distribution and reliability-first UX.
Svix	Enterprise-grade webhook sending is a mature category; developer setup and security remain active product areas.	Focus on a narrower ICP and easier entry. Do not compete on enterprise breadth initially.
Convoy	Open-source + cloud + self-hosted is a credible model; current public pricing includes Community at $0 and Premium self-hosted at $999/month.	Use open source strategically, but differentiate through onboarding, provider integrations, and tenant-aware reliability.
ApárHub	India-first webhook reliability is a viable positioning, especially around payment providers.	Avoid being “ApárHub but cheaper.” Consider broader SaaS event reliability while keeping India-native integrations.
DIY BullMQ / Redis	The biggest substitute for early-stage customers.	Make build-vs-buy economics obvious: less code, fewer failure modes, faster debugging, less operational ownership.

Competitive references are based on the current product/marketing pages reviewed on 26 August 2026. The category and feature set move quickly; pricing should be re-checked before launch.

5. Target Customers & Personas

Persona	Context	Pain	Why they buy
Founder / CTO	Owns reliability and engineering budget.	Engineering time is being spent maintaining webhook infrastructure.	Buy back engineering time and reduce customer-impacting failures.
Backend Engineer	Owns integrations, queues and webhooks.	Retries, idempotency, debugging and replay are tedious.	Fast API/SDK and excellent delivery visibility.
Platform Engineer	Operates multi-tenant systems.	Noisy tenants, rate limits, backpressure and queue isolation.	Controls, observability and tenant isolation.
Support / Ops	Gets customer tickets after integration failures.	Cannot see which event failed or why.	Searchable event timeline and self-service replay.

5.1 Primary ICP

- 5–100 employees; typically 5–20 engineers.
- Has at least one revenue-critical or customer-facing webhook flow.
- Uses Stripe/Razorpay/Shopify/GitHub/Twilio or similar event sources.
- Already has Redis, a queue, cron/background jobs, or custom retry logic—or is approaching that complexity.
- Has meaningful cost from incidents or engineering maintenance.
- Can buy a developer infrastructure product with a self-serve or light sales process.

6. Primary Use Cases

Use case	Input	Expected behavior	Priority
Outbound customer webhooks	Application event → Zyvan API	Persist, queue, sign, deliver, retry, record outcome.	P0
Inbound provider webhooks	Provider → Zyvan endpoint	Acknowledge quickly, validate signature, persist, then forward/process asynchronously.	P0
Failed endpoint recovery	Endpoint 500/timeout	Retry according to policy; preserve attempt history; stop when terminal.	P0
Manual replay	Operator selects failed event	Replay safely with new attempt ID and audit trail.	P0
Tenant throttling	Tenant produces a spike	Cap concurrency/rate and queue excess work without affecting unrelated tenants.	P0
Operational debugging	Engineer investigates incident	Search by event, tenant, endpoint, status, time; inspect timeline.	P0
Provider signature verification	Razorpay/Stripe/GitHub/etc.	Verify authenticity before accepting the event.	P0/P1
Routing/filtering	Event source → multiple destinations	Route selected events to selected endpoints.	P1
Transformations	Payload shape differs from destination	Apply safe transformations before delivery.	P2
Customer-facing portal	Zyvan customer wants their customers to inspect delivery	Hosted portal for endpoint/delivery status.	P2

7. Reliability Model & Guarantees

7.1 Delivery semantics

Zyvan should explicitly promise at-least-once delivery, not exactly-once HTTP delivery. Exactly-once effects cannot be guaranteed across an external network when a destination may process a request but fail to return a response.

- At-least-once delivery within the configured retention window.
- Durable event acceptance before asynchronous delivery work is acknowledged to the caller, subject to API contract.
- Idempotency support at ingestion and delivery layers.
- Deterministic retry scheduling with exponential backoff and jitter.
- No silent deletion of events during the active retention window.
- Explicit terminal states: delivered, expired, permanently failed, or cancelled.

7.2 Event lifecycle

1. Authenticate and validate the request.
2. Generate or validate an idempotency key/event identifier.
3. Persist event metadata and payload durably.
4. Enqueue a delivery job.

5. Worker claims the job using concurrency controls.
6. Build and sign the outgoing request.
7. Send with connection/read timeout policies.
8. Record response status, latency and attempt outcome.
9. Retry on configured transient failures.
10. Move to dead-letter/terminal state after exhaustion or expiry.
11. Allow authorized replay to create a new delivery attempt.

8. Functional Requirements

8.1 Ingestion API

Requirement	Priority	Acceptance criteria
FR-001 — Event ingestion Expose authenticated API for creating events with event type, tenant/project, payload, metadata and optional idempotency key.	**P0**	Valid request returns a durable event ID; duplicate idempotency key does not create a second logical event.

Requirement	Priority	Acceptance criteria
FR-002 — Fast acknowledgement Separate ingest from delivery so the ingestion path performs minimal synchronous work.	**P0**	Under normal conditions, API acknowledges after durable acceptance without waiting for destination completion.

Requirement	Priority	Acceptance criteria
FR-003 — Payload limits Enforce configurable maximum request and payload sizes.	**P0**	Oversized payloads are rejected with deterministic error code and do not enter the delivery queue.

8.2 Idempotency & deduplication

Requirement	Priority	Acceptance criteria
FR-010 — Idempotency key Accept caller-supplied idempotency key scoped to project/tenant.	**P0**	Same key + scope resolves to same logical event; concurrent duplicate requests do not create duplicates.

Requirement	Priority	Acceptance criteria
FR-011 — Provider event deduplication Store provider event IDs where available.	**P0**	Duplicate provider delivery is recognized and handled without duplicate logical processing.

8.3 Queue & worker system

Requirement	Priority	Acceptance criteria
FR-020 — Durable job state Persist delivery job and attempt state independently of worker process memory.	**P0**	Worker restart does not lose queued work.

Requirement	Priority	Acceptance criteria
FR-021 — Tenant concurrency Apply maximum concurrency per tenant/project/destination.	P0	Load test demonstrates one tenant cannot consume all worker slots.

Requirement	Priority	Acceptance criteria
FR-022 — Rate limiting Support delivery rate limits per destination.	P1	Configured ceiling is respected; excess events remain queued.

Requirement	Priority	Acceptance criteria
FR-023 — Backpressure Expose queue depth and delay; prevent uncontrolled worker saturation.	P1	System remains responsive during downstream slowdown.

8.4 Retry engine

Requirement	Priority	Acceptance criteria
FR-030 — Retry policy Support configurable retry count/window and backoff with jitter.	P0	Transient failures are retried; non-retriable 4xx codes follow policy without unnecessary retries.

Requirement	Priority	Acceptance criteria
FR-031 — Circuit breaking Pause or reduce delivery to repeatedly failing destinations.	P1	A destination with repeated failures is throttled/paused according to policy and can recover automatically or manually.

8.5 Dead-letter & replay

Requirement	Priority	Acceptance criteria
FR-040 — Dead-letter state Persist terminal failed events with reason and full attempt history.	P0	Failed events remain inspectable until retention expiry.

Requirement	Priority	Acceptance criteria
FR-041 — Replay event Authorized operator can replay an event to the original or selected endpoint.	P0	Replay creates a new attempt lineage without mutating historical attempts.

Requirement	Priority	Acceptance criteria
FR-042 — Bulk retry Retry a filtered set of failures.	P1	Bulk operation is rate-limited, audited and safe to stop.

8.6 Delivery security

Requirement	Priority	Acceptance criteria
FR-050 — HMAC signing Sign outbound payloads with HMAC-SHA256 and timestamp/nonce metadata.	P0	Consumer can verify signature using documented algorithm and secret.

Requirement	Priority	Acceptance criteria
FR-051 — Secret management Encrypt provider/destination secrets at rest and restrict access.	P0	Secrets are not returned in logs or dashboard payloads.

Requirement	Priority	Acceptance criteria
FR-052 — Replay protection Support timestamp tolerance and unique delivery IDs.	P1	Consumer can reject stale/replayed requests based on documented headers.

8.7 Dashboard

Requirement	Priority	Acceptance criteria
FR-060 — Event explorer Search by event ID, tenant, endpoint, event type, status and time.	P0	Operator can find an event in ≤3 interactions using common filters.

Requirement	Priority	Acceptance criteria
FR-061 — Delivery timeline Show each attempt, status, response code, latency, retry delay and outcome.	P0	Engineer can reconstruct a failure without server logs.

Requirement	Priority	Acceptance criteria
FR-062 — Replay controls Expose safe replay action with confirmation.	P0	Replay requires authorization and produces an audit entry.

Requirement	Priority	Acceptance criteria
FR-063 — Operational metrics Show throughput, success rate, queue depth, latency and retry rate.	P1	Metrics update frequently enough for incident triage.

9. API & SDK Requirements

9.1 Example API model

```
POST /v1/events

{
  "type": "invoice.paid",
  "tenant_id": "tenant_123",
  "idempotency_key": "invoice_789_paid",
  "data": { "invoice_id": "inv_789" }
}

→ 202 Accepted
{ "event_id": "evt_01J...", "status": "queued" }
```

Resource	Required fields	Notes

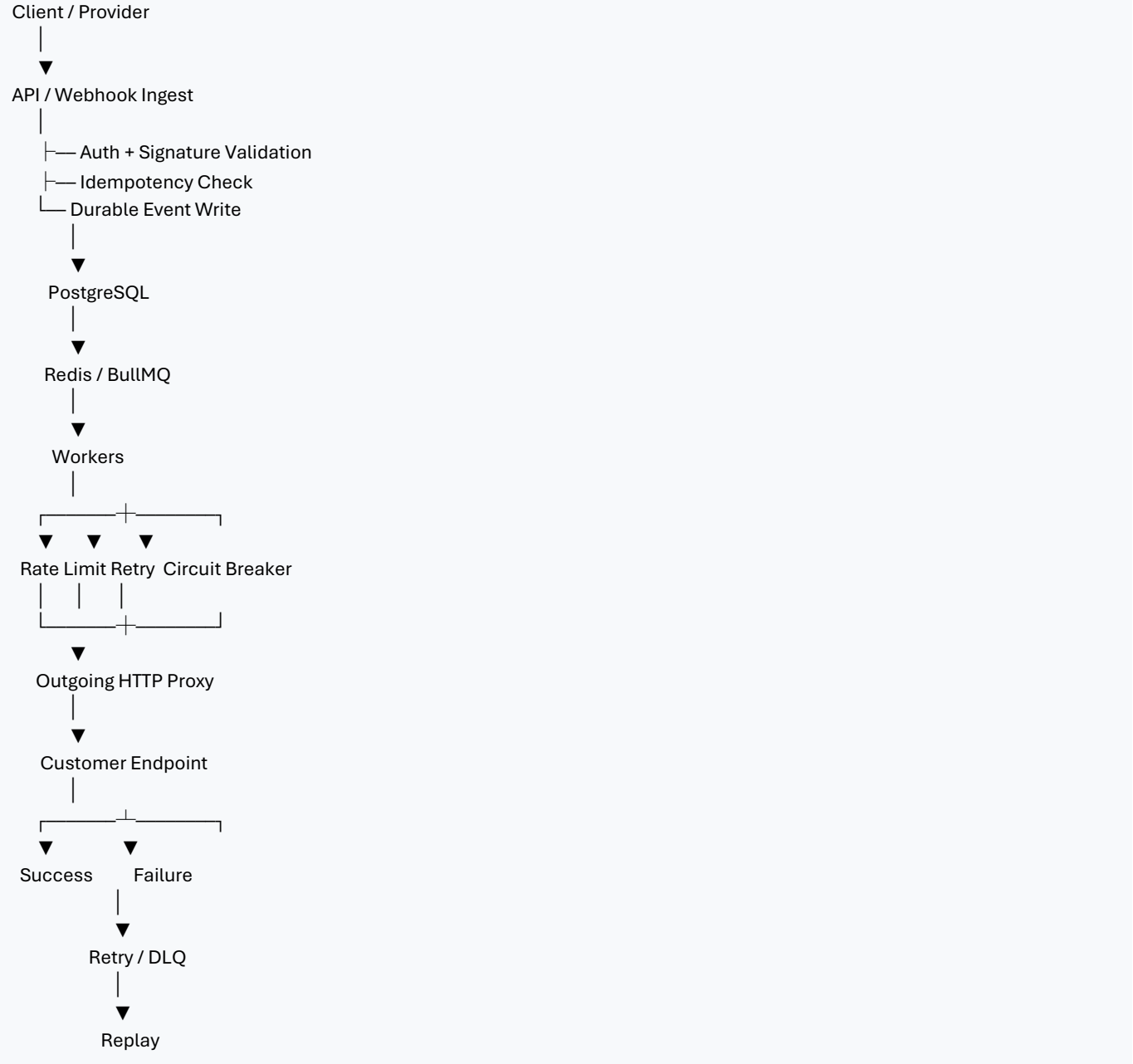
Resource	Required fields	Notes
Event	event_id, type, tenant_id, created_at, payload_ref	Payload may be stored separately from metadata at scale.
Destination	destination_id, tenant_id, URL, secret_ref, retry_policy	Secret stored encrypted; URL validation enforced.
Delivery	delivery_id, event_id, destination_id, status	Represents a logical delivery to one destination.
Attempt	attempt_id, delivery_id, started_at, ended_at, response	Immutable operational history.
Replay	replay_id, event_id, initiated_by, created_at	Never overwrite original attempt history.

9.2 SDK priorities

- JavaScript/Node.js first.
- Python second.
- curl examples for zero-install onboarding.
- Framework examples for Express/FastAPI/Next.js where useful.
- Typed event and response models where the SDK language supports them.

10. Technical Architecture

Reference architecture for MVP:



10.1 Storage principles

- PostgreSQL is the source of truth for durable event/delivery state.
- Redis/BullMQ is the execution layer, not the system of record.
- Avoid storing unlimited raw payloads in hot relational storage; introduce object storage for large/old payloads when required.
- Partition high-volume event/attempt tables before they become operationally painful.
- Define retention independently for payloads, metadata and metrics.

10.2 Failure modes to test

Failure	Expected result
API process dies after database commit	Event remains durable and is eventually queued/reconciled.
Worker crashes mid-attempt	Job becomes retryable without duplicate logical delivery state corruption.
Redis restarts	PostgreSQL remains source of truth and queued work can be reconstructed/recovered.

Failure	Expected result
Destination returns 500	Retry according to policy.
Destination times out after processing	Retry may occur; consumer deduplication/idempotency semantics prevent duplicate business effect.
One tenant spikes 100×	Per-tenant limits protect other tenants.
Database is temporarily unavailable	Ingress fails explicitly rather than falsely acknowledging durable acceptance.
Secret is rotated	New secret used for subsequent delivery without exposing old secret.

11. Security & Compliance

- TLS for all external API and delivery traffic.
- HMAC-SHA256 signing for outbound webhooks with documented signature versioning.
- Encrypt secrets at rest; avoid plaintext secrets in logs, traces and support exports.
- API keys should support rotation, scoped permissions and immediate revocation.
- SSRF protections for outbound HTTP: private IP ranges, localhost, link-local and cloud metadata endpoints blocked by default; explicit allow-listing for exceptional enterprise use cases.
- Payload access should be permission-controlled and auditable.
- Retention settings must be explicit; raw payload retention should be minimized where possible.
- SOC 2, SSO/SAML, SCIM, dedicated infrastructure and formal SLAs are post-PMF enterprise roadmap items, not MVP blockers.

12. Observability & SRE Requirements

Metric	Purpose	Initial target
Ingestion success rate	Detect API/storage failures	≥ 99.9% monthly excluding customer-side invalid requests
Delivery success rate	Measure webhook health	Dashboard by destination and tenant
Retry rate	Detect downstream instability	Track trend; alert on abnormal deviation
Queue age	Detect backlog / backpressure	Alert on sustained threshold breach
Delivery latency p50/p95/p99	Protect user experience	Track by destination/tenant
Worker utilization	Capacity planning	Maintain headroom; avoid sustained saturation
DLQ growth	Detect persistent failures	Alert immediately for revenue-critical workflows

12.1 Customer-facing alerts

- Destination failure threshold exceeded.
- Queue backlog / delivery delay threshold exceeded.
- Endpoint repeatedly timing out.
- DLQ count above threshold.
- Signature validation failures above threshold.
- Usage nearing plan limit.

13. Monetization & Pricing Strategy

Pricing should be treated as a hypothesis until real traffic benchmarks are available. The core unit is a delivered event, but the business must also protect against payload size, retries, retention and unusually high throughput.

Plan	Illustrative price	Included usage	Retention	Purpose
Free	₹0/month	10K events/month	3–7 days	Evaluation and developer adoption
Developer	₹999/month	100K events/month	14–30 days	Individual developer / small app
Startup	₹2,999/month	1M events/month	30 days	Growing SaaS
Growth	₹9,999/month	10M events/month	90 days	Multi-tenant / higher volume
Enterprise	₹25K–₹2L+/month	Custom	Custom	Compliance, SLA, dedicated capacity

13.1 Pricing guardrails

- Never offer unlimited events on low-priced plans.
- Define maximum payload size per plan.
- Charge or gate additional retention and high throughput.
- Use overages rather than hard failure when appropriate for production accounts.
- Track cost per 1K / 100K / 1M events before finalizing prices.
- Target gross margin ≥ 75% after infrastructure + variable service costs at steady state; revisit if margins are consistently below 60%.
- Do not use competitor price alone to set Zyvan pricing.

14. Unit Economics & Financial Model

The main cost drivers are compute, PostgreSQL storage/IO, Redis memory/compute, outbound bandwidth, log/telemetry volume, object storage, backups, email/notification services and payment processing. Retry volume multiplies delivery traffic, so pricing must be based on delivered/processed volume with explicit limits.

Scenario	Customers	ARPA	MRR	ARR
Validation	10	₹2,500	₹25,000	₹3,00,000
Early PMF	50	₹3,000	₹1,50,000	₹18,00,000
Traction	100	₹4,000	₹4,00,000	₹48,00,000
Strong SMB SaaS	250	₹5,000	₹12,50,000	₹1.5 Cr
Scaled	500	₹6,000	₹30,00,000	₹3.6 Cr

These are planning scenarios, not forecasts. The first business proof point is not a large ARR number; it is 3–5 production customers paying for the product and continuing to use it.

15. UX & Dashboard Requirements

Screen	Must show	Primary action
Overview	Events today, delivery success, retries, queue age, DLQ, usage	Drill into incident
Events	Search/filter by event ID, type, tenant, destination, status, date	Open event
Event detail	Payload, headers, delivery timeline, attempt history, retry schedule	Replay / copy diagnostics

Screen	Must show	Primary action
Destinations	URL, status, signing key status, retry policy, rate limit	Pause / edit / test
Tenants	Per-tenant volume, success rate, concurrency, queue depth	Adjust controls
Alerts	Current incidents and thresholds	Acknowledge / resolve
Settings	API keys, members, retention, billing	Configure account

16. Roadmap

Phase 0 — Validation foundation (0–4 weeks)

- Harden ingestion, idempotency, retry engine, DLQ and replay.
- Build searchable event/delivery dashboard.
- Add Node.js SDK + curl examples.
- Instrument cost and throughput benchmarks.
- Create 3–5 provider signature adapters (start with Shopify, Stripe, Razorpay, GitHub, Cashfree).
- Document reliability semantics clearly.

Phase 1 — Production MVP (4–8 weeks)

- Per-tenant concurrency controls.
- Destination rate limiting and backpressure.
- Circuit breaker.
- Alerting for failures/backlog.
- Basic billing and usage metering.
- Public status page and operational runbooks.
- Onboard 10 design partners.

Phase 2 — Product-market signals (8–16 weeks)

- Convert at least 3 design partners to paid production use.
- Add team/RBAC basics.
- Improve self-serve onboarding.
- Add routing/filters only where real customers request them.
- Launch India-region option if demand is demonstrated.
- Refine pricing based on actual event economics.

Phase 3 — Scale / moat (post-PMF)

- Object-storage event archival.
- Advanced transformations.
- Customer-facing portal.
- Terraform provider / CLI.
- Multi-region deployment.
- SSO/SAML/SCIM, audit logs and formal compliance program.
- Optional self-hosted/core open-source packaging.

17. Product & Business Success Metrics

Metric	Target / decision rule
Time to first event	<15 minutes for a technical user without support
Activation	≥40% of qualified trials send at least one successful event
Production conversion	≥20% of qualified design partners reach production
Paid conversion	≥30% of active production accounts pay within 60 days
Retention	Track logo retention and event volume retention; investigate churn drivers monthly
Gross margin	Target ≥75% at scale; never hide infrastructure overages
Support burden	<1 hour/month average support per SMB account once stabilized
Reliability incidents	Every internal incident produces a blameless RCA and corrective action
Revenue milestone #1	₹10K MRR
Revenue milestone #2	₹50K MRR
Revenue milestone #3	₹1L MRR

18. Analytics Instrumentation

Event	When fired	Key properties
account_created	Workspace created	source, plan
api_key_created	First credential created	project_id
first_event_ingested	First event accepted	event_type, provider
first_delivery_success	First destination success	destination_type
first_replay	First replay initiated	operator_type
production_signal	Sustained real traffic	events_7d, endpoints
upgrade_started	Billing upgrade	from_plan, to_plan
upgrade_completed	Payment succeeds	plan, amount
churned	Account cancels or becomes inactive	plan, reason

19. Key Risks & Mitigations

Risk	Severity	Mitigation
Feature parity race with Hookdeck/Svix/Convoy	High	Narrow ICP and reliability-first wedge; do not chase breadth before traction.
DIY BullMQ is “good enough”	High	Show total engineering cost and provide zero-to-production onboarding.
Infrastructure costs exceed revenue	High	Benchmark event economics; usage limits; overages; payload/retention caps.
Security incident	Critical	SSRF prevention, secrets management, HMAC, audit logs, least privilege.
One tenant starves others	High	Per-tenant concurrency, rate limits, queue isolation and backpressure.
Users do not trust a young reliability vendor	High	Open source/core transparency, clear incident history, status page, docs, self-hosted path later.
No repeatable acquisition	High	Start with communities where the pain is visible: Shopify/SaaS/fintech developers.
Overbuilding before PMF	High	Feature freeze after MVP; prioritize customer requests backed by production use.

20. MVP Acceptance Criteria

12. A developer can create a project and send a test event via curl in under 15 minutes.
13. The event is durably persisted before the API claims successful acceptance.
14. The same idempotency key cannot create duplicate logical events under concurrent requests.
15. A 500/timeout sequence demonstrates automatic retry with jitter and persistent attempt history.
16. A terminally failed event is visible in a DLQ and can be replayed without overwriting original history.
17. A destination can be paused and resumed safely.
18. Per-tenant concurrency limits prevent one tenant from consuming all available worker slots in load tests.
19. Outbound requests use documented HMAC signatures and never expose secrets in logs.
20. Dashboard can find an event and reconstruct its full delivery timeline without server access.
21. Usage metering reports events, retries, retention usage and plan utilization accurately enough for billing.
22. The system has automated tests for duplicate deliveries, out-of-order events, worker crash recovery, retry exhaustion, secret rotation and queue backlog.
23. At least 3 external companies run real production traffic through Zyvan before expanding scope materially.

21. Go-to-Market Validation Plan

21.1 First customer acquisition motion

24. Identify 50 Shopify/SaaS/fintech teams that visibly operate webhooks or background-job infrastructure.
25. Approach with a technical problem statement, not a generic product pitch.
26. Offer a free design-partner period in exchange for production feedback and permission to use anonymized reliability data.
27. Measure whether they migrate one real webhook flow from DIY infrastructure.
28. Convert successful production users to a paid plan based on volume and operational value.

21.2 Validation questions

- What webhook/event failure has caused a customer-facing incident recently?

- What does your current retry/idempotency system look like?
- How much engineering time is spent maintaining it?
- What would make you trust an external provider with production events?
- Would you migrate one production workflow if setup took <15 minutes?
- What monthly price feels cheaper than maintaining it yourself?

22. Definition of Done for v1.0

- All P0 requirements implemented and covered by automated tests.
- Basic P1 reliability controls implemented where required for safe production use.
- Load tests completed at 100K and 1M event scenarios with documented cost/performance results.
- Security review completed for SSRF, secrets, authentication, authorization and payload handling.
- Public documentation includes API reference, reliability semantics, retry behavior, HMAC verification and troubleshooting.
- Status page and incident communication process exist.
- At least 3 production customers have completed a successful integration or are actively running production traffic.
- Pricing is based on measured cost per event, not copied from competitors.

23. Final Product Decision

BUILD — but do not build a generic Hookdeck/Svix clone.

Zyvan should focus the first release on: durable event ingestion, at-least-once delivery, idempotency, retries, dead-letter recovery, replay, delivery debugging, tenant isolation, and a very fast developer onboarding path.

The product should earn the right to expand only after 3–5 paying production customers demonstrate that reliability is painful enough to buy.

24. Competitive Reference Sources

- Hookdeck Event Gateway / Pricing: https://hookdeck.com/pricing
- Hookdeck Event Gateway: https://hookdeck.com/event-gateway
- Convoy Pricing: https://www.getconvoy.io/pricing
- Convoy Self-Hosted: https://www.getconvoy.io/self-hosted
- Svix: https://www.svix.com/
- AparHub: https://aparhub.com/

Note: Competitive references were reviewed on 26 August 2026. The PRD intentionally treats competitor features and pricing as moving inputs rather than permanent assumptions.

25. Technology Stack & Engineering Standards

The technology stack is selected to keep the MVP operationally simple while preserving a path to higher scale. The MVP should prefer proven components over premature platform complexity.

Layer	MVP choice	Purpose	Production rule
Web application	Next.js (React) + TypeScript	Dashboard, onboarding, account/project management	Keep server actions thin; core delivery remains in dedicated API/worker services.
API	Node.js + TypeScript (Fastify recommended)	Ingestion API, management API, auth and usage endpoints	Stateless API instances; request validation and rate limits at the edge.
SDK	TypeScript/Node.js first; Python second	Developer integration	Generated/open-source API types where practical; minimal surface area.
Primary database	PostgreSQL	Source of truth for events, deliveries, attempts, tenants, projects, keys and billing state	Use indexes, partitioning and connection pooling as volume grows.
Queue / execution	Redis + BullMQ	Asynchronous job execution, scheduling, concurrency control and delayed retries	Redis is not the system of record; jobs must be reconstructable from PostgreSQL.
Object storage	S3-compatible object storage (provider-agnostic)	Cold payload/archive storage and large payloads	Keep large/old payloads out of hot PostgreSQL storage.
Caching	Redis	Short-lived caches, rate-limit counters and queue coordination	Cache misses must never cause data loss.
Authentication	API keys for service access; email/OIDC-ready account auth	Secure machine-to-machine and dashboard access	Support key rotation, scopes and revocation.
Webhook security	HMAC-SHA256	Outbound signature verification	Version signatures and prevent secrets from entering logs.
Validation	Zod or equivalent schema validation	Request/response validation	Reject malformed requests deterministically.
Observability	OpenTelemetry + structured JSON logs + Prometheus-compatible metrics	Traces, metrics and logs	Track event latency, queue depth, attempt outcomes and tenant saturation.
Frontend hosting	Vercel or equivalent	Dashboard delivery	Separate dashboard failure from delivery-plane availability.
Compute	Containerized Linux services	API, workers and scheduled jobs	Use Docker; keep deployment portable.
CI/CD	GitHub Actions	Build, test, security scanning and deployment	Block release on failing reliability/security suites.
Testing	Vitest/Jest + Supertest + Playwright + load testing tooling	Unit, API, integration, E2E and load tests	Failure-mode tests are mandatory, not optional.
Payments	Razorpay/Stripe billing abstraction	Subscription and usage billing	Do not couple core event processing to payment provider APIs.
Error tracking	Sentry or equivalent	Application exceptions and frontend errors	Never put raw payloads/secrets into error context.
Infrastructure as code	Terraform/OpenTofu post-MVP	Repeatable cloud environments	Introduce after the core production topology stabilizes.

25.1 Logical deployment topology

Separate the control plane from the delivery plane conceptually even if they share infrastructure during MVP.

- Control plane: dashboard, projects, tenants, destinations, API keys, billing, policies and configuration.
- Ingestion plane: authenticated event intake and durable persistence.
- Delivery plane: queue consumers, retry scheduler, outbound HTTP client, concurrency and rate controls.
- Observation plane: metrics, traces, logs, incident alerts and customer-visible delivery history.
- Storage plane: PostgreSQL for hot durable state and object storage for large/cold payloads.

25.2 Technology principles

- PostgreSQL is authoritative for event and delivery state; Redis/BullMQ is an execution mechanism.
- Every asynchronous operation must be recoverable after worker or process loss.
- All external HTTP calls have explicit connect, read and total timeout behavior.
- Reliability features must be tenant-aware so one customer cannot exhaust global capacity.
- Any technology choice that increases operational burden must have a measurable product or reliability benefit.

26. UX Design Specification

The UX should make reliability state understandable within seconds. The primary user is an engineer debugging a production event, not a general business user.

26.1 UX principles

- Show the current state first; expose implementation details second.
- Make failure recovery a first-class action: retry, replay, pause, resume and inspect.
- Use consistent terminology: Event, Destination, Attempt, Delivery, Retry, DLQ, Tenant.
- Never hide uncertainty. Distinguish queued, delivering, retrying, delivered, expired and permanently failed states.
- Avoid dashboard decoration that does not help diagnose reliability or usage.

26.2 Primary information architecture

Area	Primary purpose	Key screens
Overview	Operational health	Delivery rate, failures, backlog, latency, usage
Events	Find and inspect events	Search, filters, event detail, timeline, payload
Destinations	Manage delivery endpoints	Endpoint config, status, retries, limits, pause/resume
Tenants	Control multi-tenant behavior	Tenant list, usage, concurrency, rate limits
Dead Letter	Recover failed events	DLQ queue, failure reason, replay, bulk actions
Developers	Integrate Zyvan	API keys, SDKs, code examples, webhook signing
Settings	Configure project	Retention, retry policies, alerts, team access
Billing	Understand cost	Plan, usage, overages, invoices, limits

26.3 Core user flows

29. Create account → create project → copy API key → send first event with curl → see event in dashboard.
30. Create destination → configure URL and secret → send test event → verify signature → enable production delivery.
31. Open failed event → inspect timeline and response → view retry schedule → replay → confirm new attempt.
32. Open noisy tenant → inspect throughput/concurrency → lower tenant limit or pause destination → observe recovery.
33. Open billing → review included events, retries and retention → upgrade or enable overage handling.

26.4 Event detail screen

- Header: event ID, type, tenant, current status, created time.
- Timeline: receive, persist, enqueue, attempt, retry scheduling and final state.
- Delivery card: destination, HTTP method, latency, status code, attempt number and next retry time.
- Request inspection: headers excluding secrets, payload preview with redaction controls.
- Recovery actions: retry now, replay, cancel, copy event ID, create support link.
- Failure explanation: human-readable reason plus machine-readable error category.

26.5 UX states

State	Visual meaning	Allowed actions
Queued	Accepted and waiting	View, cancel if policy permits
Delivering	Worker actively sending	View attempt
Retrying	Transient failure; next attempt scheduled	Retry now, pause destination
Delivered	Successful terminal delivery	View history, replay
Dead letter	Retry policy exhausted or manually moved	Replay, inspect, export
Expired	Retention window ended	View metadata if retained; no replay if payload deleted
Cancelled	Explicitly stopped	Review history; replay if policy permits

26.6 Accessibility and usability

- Keyboard-accessible primary actions and filters.
- Do not encode status only with color; pair color with labels/icons.
- Responsive dashboard for laptop-first use, with mobile support for alerts and incident review.
- Use plain-language error explanations beside raw HTTP/error data.
- Loading, empty, partial-failure and permission-denied states must be designed explicitly.

27. API Design Specification

The public API is the product. It must be predictable, versioned, idempotent, observable and safe to automate against.

27.1 API conventions

- Base URL: https://api.zyvan.dev/v1
- JSON request and response bodies.
- Authentication: Bearer API key for server-to-server calls.
- Use UUID/ULID-style opaque identifiers; do not expose sequential database IDs.
- ISO 8601 UTC timestamps.
- Idempotency-Key header supported on event creation.
- Pagination with cursor-based tokens for event/delivery collections.
- Stable machine-readable error codes plus human-readable messages.
- API versioning through the URL; backward-compatible additions within a version.

27.2 Resource model

Resource	Purpose	Key fields

Resource	Purpose	Key fields
Project	Isolation boundary for configuration and usage	project_id, name, plan, status
Tenant	Customer/merchant/account receiving isolated controls	tenant_id, external_id, limits, status
Destination	Outbound HTTP target	destination_id, URL, tenant_id, secret_ref, policy
Event	Durable logical event	event_id, type, tenant_id, payload_ref, created_at, status
Delivery	Logical delivery to a destination	delivery_id, event_id, destination_id, status
Attempt	One HTTP delivery attempt	attempt_id, delivery_id, attempt_no, status_code, latency_ms
API Key	Machine access credential	key_id, scopes, created_at, revoked_at
Replay	Recovery action derived from an existing event	replay_id, source_event_id, requested_by, status

27.3 Core endpoints

Method	Endpoint	Purpose	Expected result
POST	/v1/events	Create event	202 Accepted with event_id
GET	/v1/events/{event_id}	Inspect event	Event state + delivery summary
GET	/v1/events	Search events	Cursor-paginated list
POST	/v1/events/{event_id}/replay	Replay event	202 Accepted with replay_id
POST	/v1/destinations	Create destination	201 Created
GET	/v1/destinations/{destination_id}	Inspect destination	Destination config minus secrets
PATCH	/v1/destinations/{destination_id}	Update destination	200 OK
POST	/v1/destinations/{destination_id}/pause	Pause delivery	200 OK
POST	/v1/destinations/{destination_id}/resume	Resume delivery	200 OK
GET	/v1/destinations/{destination_id}/deliveries	List deliveries	Cursor-paginated list
GET	/v1/usage	Usage/limits	Current cycle usage
POST	/v1/api-keys	Create key	201 Created; key shown once
DELETE	/v1/api-keys/{key_id}	Revoke key	204 No Content

27.4 Event creation contract

Request:

```
{
  "type": "invoice.paid",
  "tenant_id": "tenant_123",
  "idempotency_key": "invoice_789_paid",
```

```
  "data": { "invoice_id": "inv_789" }
}
```

Response (success):

```
{
  "event_id": "evt_01J...",
  "status": "queued",
  "created_at": "2026-08-26T10:00:00Z"
}
```

27.5 Error contract

Every API error should return: code, message, request_id and optional details. Example categories: invalid_request, authentication_failed, authorization_denied, duplicate_idempotency_key, rate_limited, not_found, conflict, payload_too_large, internal_error.

27.6 Delivery signature contract

- Use a versioned signature scheme such as X-Zyvan-Signature: v1=<hex> or equivalent documented format.
- Sign a canonical timestamp + raw payload representation to reduce replay risk.
- Include a delivery/event identifier in headers so consumers can implement idempotency.
- Document a recommended replay window and clock-skew tolerance.

28. Software Requirements Specification (SRS)

This SRS translates the product requirements into implementable system requirements. Requirement IDs should be used in design, development, testing and release tracking.

28.1 System actors

Actor	Responsibility
Developer/Customer	Creates projects, destinations and events; monitors delivery; replays failures.
Customer application	Publishes events to Zyvan and receives outbound webhooks.
Zyvan API	Authenticates, validates and durably accepts events and configuration changes.
Scheduler/Queue	Coordinates asynchronous execution and retry timing.
Worker	Executes delivery attempts and records outcomes.
Database	Maintains authoritative state.
Notification service	Sends operational alerts.
Billing service	Measures usage and applies plan/overage rules.

28.2 Functional requirements

ID	Area	Requirement	Priority
SRS-001	Authentication	The API shall reject requests without a valid active credential.	P0
SRS-002	Authorization	The API shall enforce project/tenant/key scopes on every protected operation.	P0
SRS-003	Durable ingestion	The system shall persist an accepted event before returning successful acceptance.	P0

ID	Area	Requirement	Priority
SRS-004	Idempotency	Concurrent requests using the same idempotency key within a project scope shall resolve to one logical event.	P0
SRS-005	Queue recovery	A worker crash shall not permanently lose an accepted delivery job.	P0
SRS-006	Retry policy	The system shall retry configured transient failures according to exponential backoff and jitter.	P0
SRS-007	Retry classification	The system shall distinguish retryable errors from terminal errors using status class, timeout and policy rules.	P0
SRS-008	Delivery timeout	Every outbound request shall enforce configurable connect/read/overall timeouts.	P0
SRS-009	Attempt history	Each attempt shall record timestamp, outcome, latency, HTTP status where available and sanitized response metadata.	P0
SRS-010	Dead letter	Exhausted/expired events shall enter a recoverable terminal state with complete history.	P0
SRS-011	Replay	Authorized users shall be able to create a new delivery attempt without deleting original history.	P0
SRS-012	Tenant isolation	Per-tenant concurrency and/or rate limits shall prevent one tenant from consuming all worker capacity.	P0
SRS-013	Destination pause	A destination can be paused without deleting queued events.	P1
SRS-014	Backpressure	Delivery above configured throughput shall be queued rather than silently discarded.	P1
SRS-015	Circuit breaker	Repeated destination failure can temporarily stop delivery attempts and later recover automatically.	P1
SRS-016	Signature	Outbound webhooks shall be signed with a documented HMAC scheme.	P0
SRS-017	Secret handling	Secrets shall be encrypted at rest and excluded from normal logs/traces.	P0
SRS-018	SSRF protection	Outbound destinations targeting unsafe private/metadata addresses shall be blocked by default.	P0
SRS-019	Observability	The platform shall emit metrics, logs and traces sufficient to reconstruct delivery health.	P0
SRS-020	Usage metering	The system shall meter billable events and relevant variable resources with auditable counters.	P1
SRS-021	Retention	The system shall enforce configurable metadata and payload retention.	P1
SRS-022	Auditability	Security-sensitive actions such as key creation/revocation and replay shall be auditable.	P1
SRS-023	Availability	The delivery plane shall be deployable with redundant instances and health checks.	P1
SRS-024	API stability	All public API responses shall conform to documented versioned schemas.	P0

28.3 Non-functional requirements

ID	Requirement	Target

ID	Requirement	Target
NFR-001	Ingestion acknowledgement latency	Target p95 < 300 ms excluding upstream network time for normal payloads.
NFR-002	Durability	No accepted event loss during single worker-process failure.
NFR-003	Delivery reliability	At-least-once semantics within configured retention window.
NFR-004	Security	TLS, least privilege, secret protection and SSRF defenses mandatory.
NFR-005	Scalability	Horizontal API/worker scaling without changing customer API contracts.
NFR-006	Observability	Every event has a traceable event_id and delivery attempt lineage.
NFR-007	Recovery	Documented restore and replay procedures for database/queue incidents.
NFR-008	Accessibility	Core dashboard flows conform to WCAG 2.1 AA intent where practical for MVP.

28.4 Data model requirements

- projects(id, account_id, name, plan, status, created_at, updated_at)
- tenants(id, project_id, external_id, concurrency_limit, rate_limit, status, created_at)
- destinations(id, project_id, tenant_id, url, secret_ref, policy_id, status, created_at)
- events(id, project_id, tenant_id, type, idempotency_key, payload_ref, status, created_at, expires_at)
- deliveries(id, event_id, destination_id, status, next_attempt_at, final_attempt_at)
- delivery_attempts(id, delivery_id, attempt_no, started_at, finished_at, outcome, status_code, latency_ms, sanitized_response)
- api_keys(id, project_id, key_hash, scopes, created_at, last_used_at, revoked_at)
- usage_counters(project_id, billing_period, events, attempts, bytes, storage_bytes)
- audit_logs(id, project_id, actor, action, resource_type, resource_id, created_at, metadata)

28.5 State machines

Event state: received → persisted → queued → delivering → delivered | retrying → dead_letter | expired | cancelled.

Destination state: active ↔ paused; active → degraded via circuit breaker; degraded → active after recovery criteria.

Replay state: requested → queued → delivering → delivered | retrying | failed.

28.6 Testing requirements

- Unit tests for retry classification, backoff calculation, HMAC signing, idempotency and policy evaluation.
- Integration tests with PostgreSQL and Redis/BullMQ for worker crash and retry recovery.
- API contract tests for all public v1 endpoints.
- End-to-end tests for create event → delivery → failure → retry → success.
- Security tests for SSRF, auth bypass, secret leakage, replay attacks and malformed payloads.
- Load tests covering 100K and 1M events with measured throughput, queue lag and infrastructure cost.
- Chaos tests for worker termination, Redis restart and downstream endpoint outage.

28.7 Release gates

34. All P0 SRS requirements pass automated tests.
35. No known critical security issue remains open.
36. Load tests meet documented throughput and durability targets.
37. Monitoring and alerting are active for ingestion, queue and delivery health.
38. A production incident runbook exists for database, queue, worker and downstream outages.
39. At least three external production integrations have completed a successful reliability test.

29. Product-to-Engineering Traceability

This matrix connects the business goals to product components and implementation artifacts so that the team can avoid building features without a measurable reason.

Business goal	Product requirement	UX/API/SRS component	Success evidence
Never lose accepted events	Durable ingestion	API-Event + SRS-003	Accepted event remains queryable after worker crash
Recover from endpoint failure	Retry + DLQ + replay	UX event timeline + SRS-006/010/011	Failure sequence eventually succeeds after recovery
Avoid noisy neighbors	Tenant isolation	UX tenant controls + SRS-012	Load test shows fair worker allocation
Reduce debugging time	Timeline + logs	UX event detail + SRS-009/019	Engineer reconstructs incident without server access
Monetize predictably	Usage metering	API usage + SRS-020	Billing counters reconcile to event ledger
Protect production systems	SSRF + HMAC + secrets	API security + SRS-016/017/018	Security tests pass

30. Recommended Repository Structure

The repository should mirror the product architecture so that API, UX and infrastructure remain separable.

```
zyvan/
 apps/
  web/            # Next.js dashboard
  api/            # public + admin APIs
  worker/         # BullMQ delivery workers
 packages/
  sdk-node/
  sdk-python/
  schemas/        # shared validation/types
  crypto/         # signing/verification helpers
 infra/
  docker/
  terraform/
 docs/
  PRD.md
  UX_SPEC.md
  API_SPEC.md
  SRS.md
  RUNBOOKS.md
 tests/
  unit/
  integration/
  e2e/
  load/
```

31. Final Recommendation

Build Zyvan as a focused reliability product, not a feature-for-feature clone of Hookdeck/Svix/Convoy. The MVP should win on fast setup, explicit at-least-once semantics, excellent failure debugging, tenant isolation and a developer-friendly India-first entry point. The next business milestone remains 3–5 production customers, not a larger feature list. Once those customers validate the core problem, the team can use this PRD, UX specification, API specification and SRS as the controlled expansion baseline.